

Snowflake Architecture and Editions

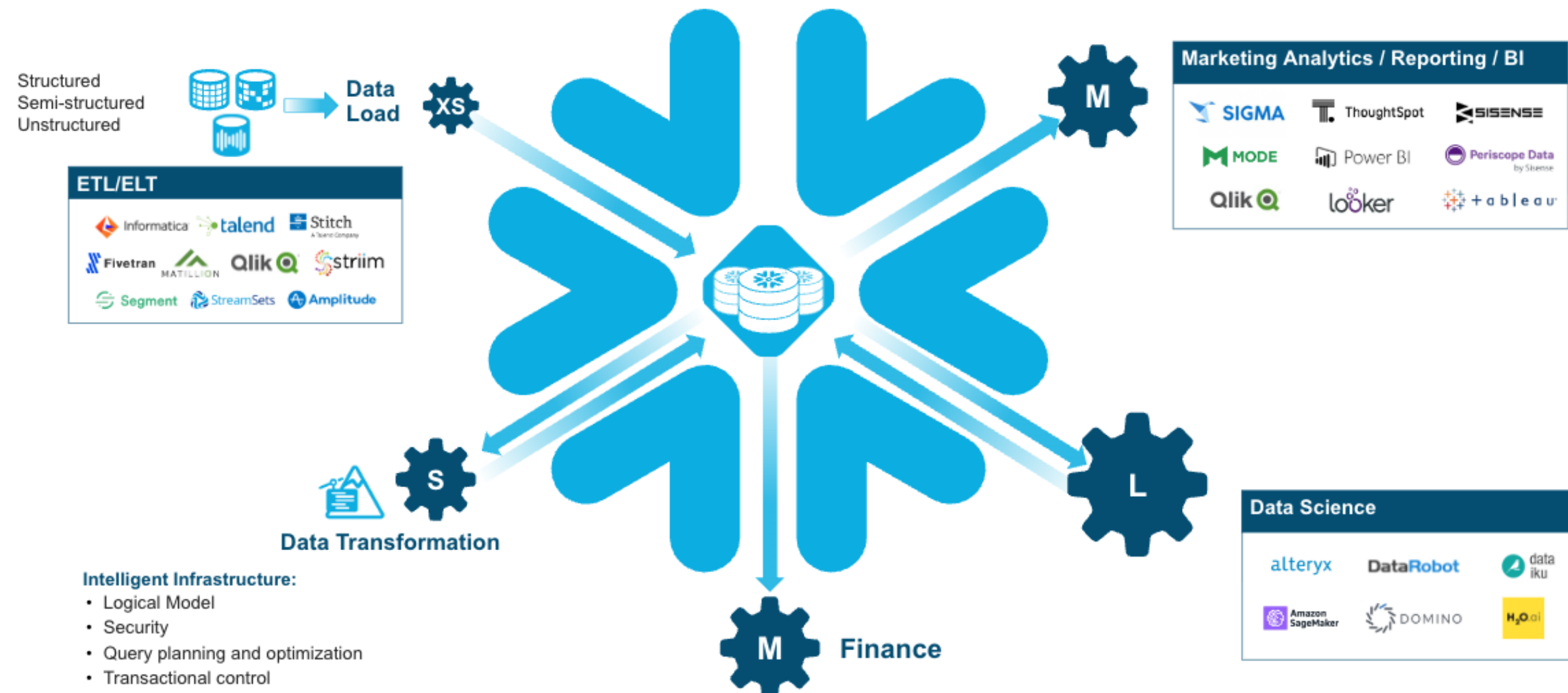
SNOWFLAKE ARCHITECTURE



Emily Melhuish

Technical Curriculum Developer,
Snowflake

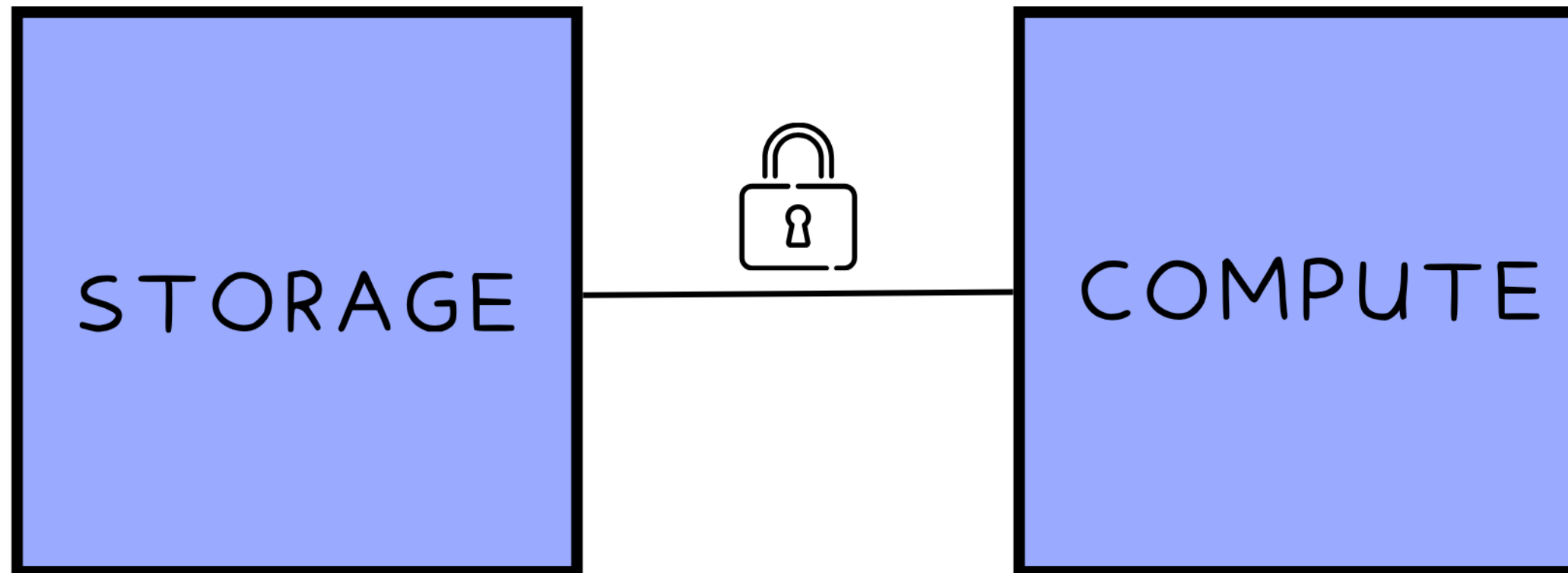
Snowflake Functional Architecture



¹ * Snowflake Learning Material

The Problem With Traditional Databases

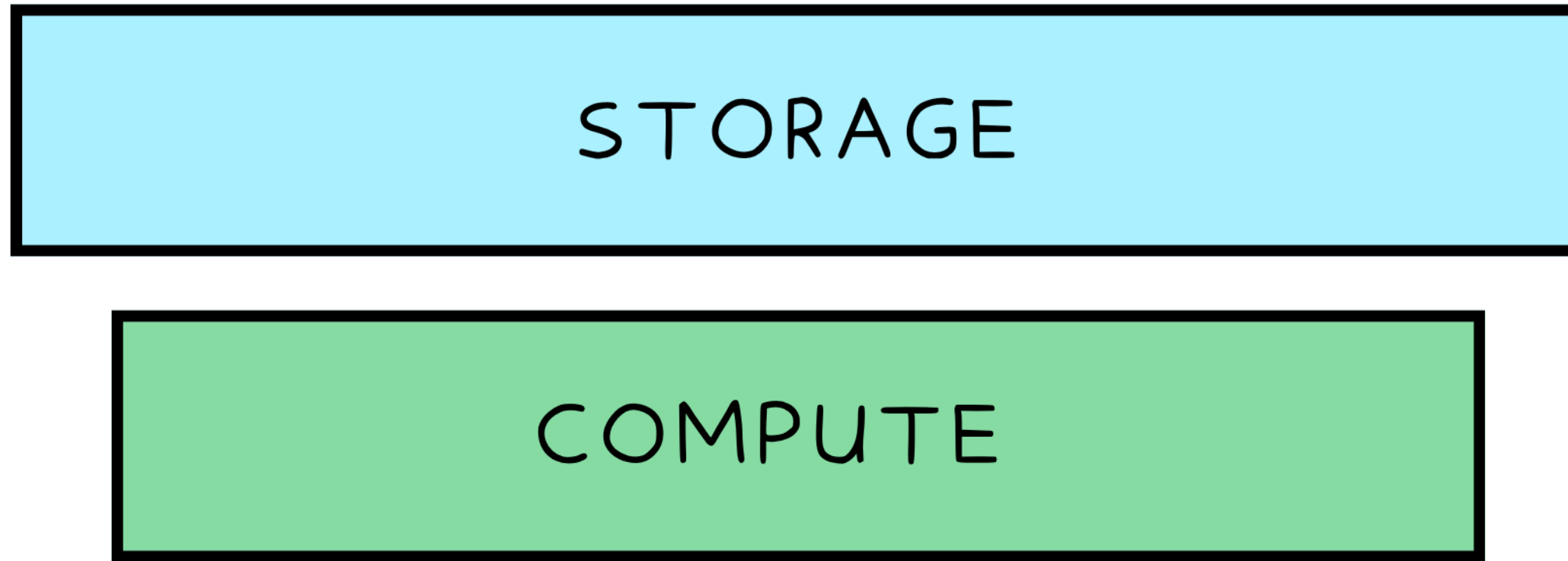
TRADITIONAL DATABASES =
TIGHTLY COUPLED



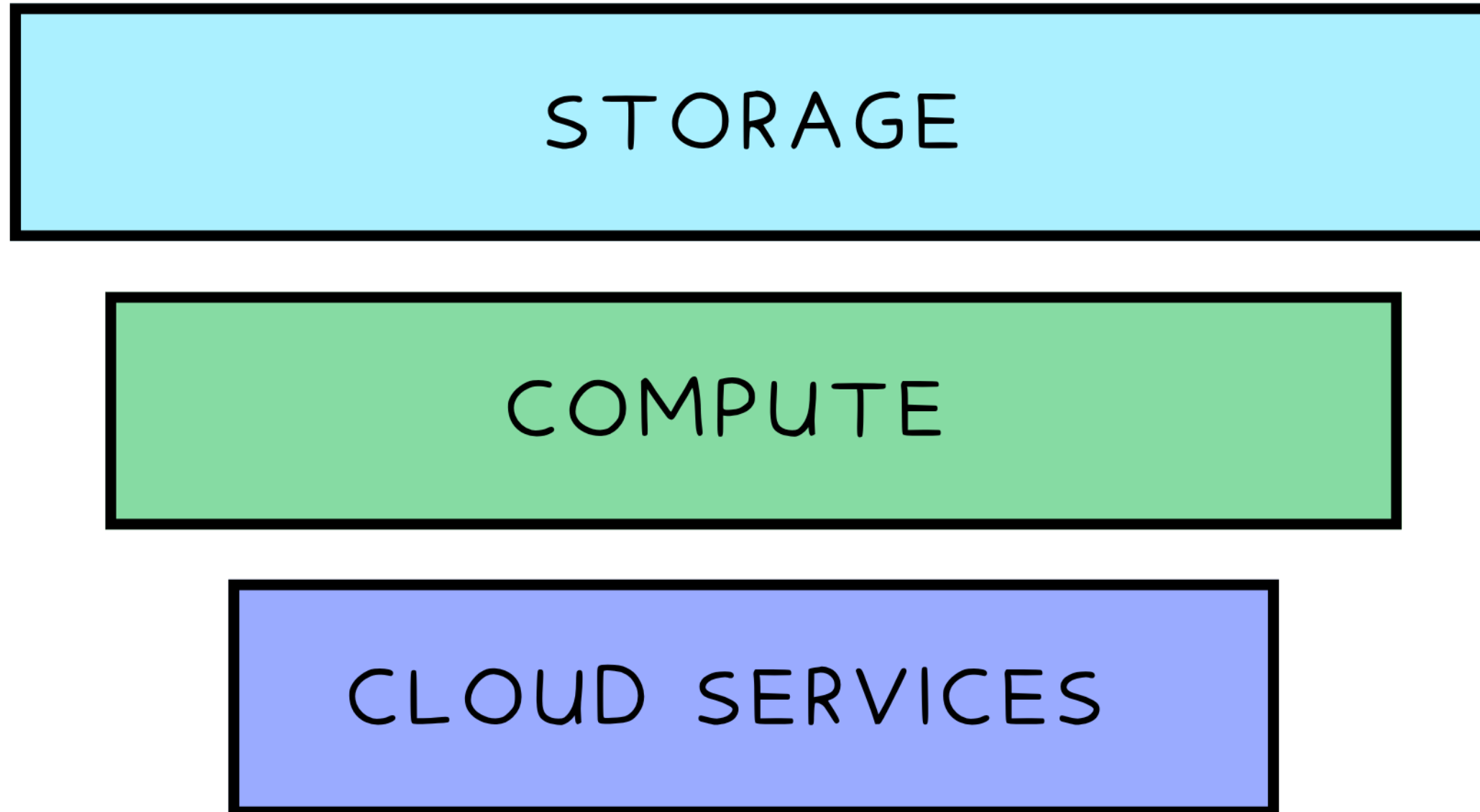
Three Layers: Storage



Three Layers: Compute

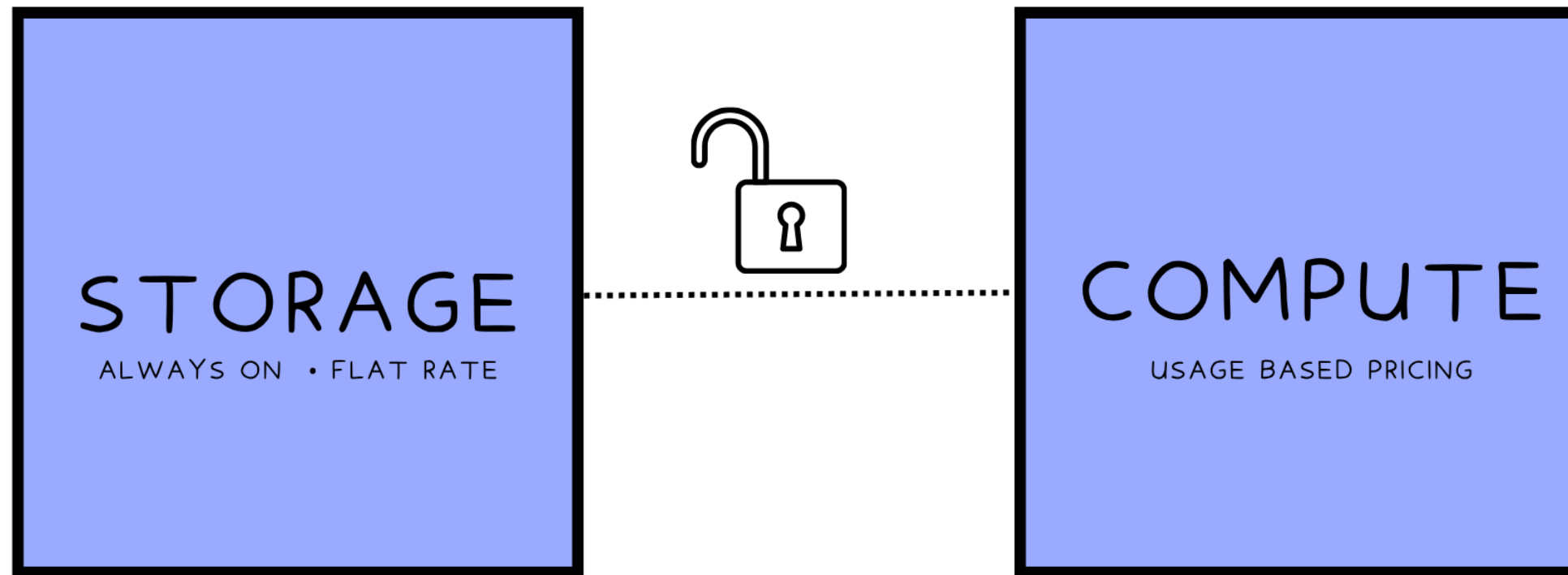


Three Layers: Cloud Services

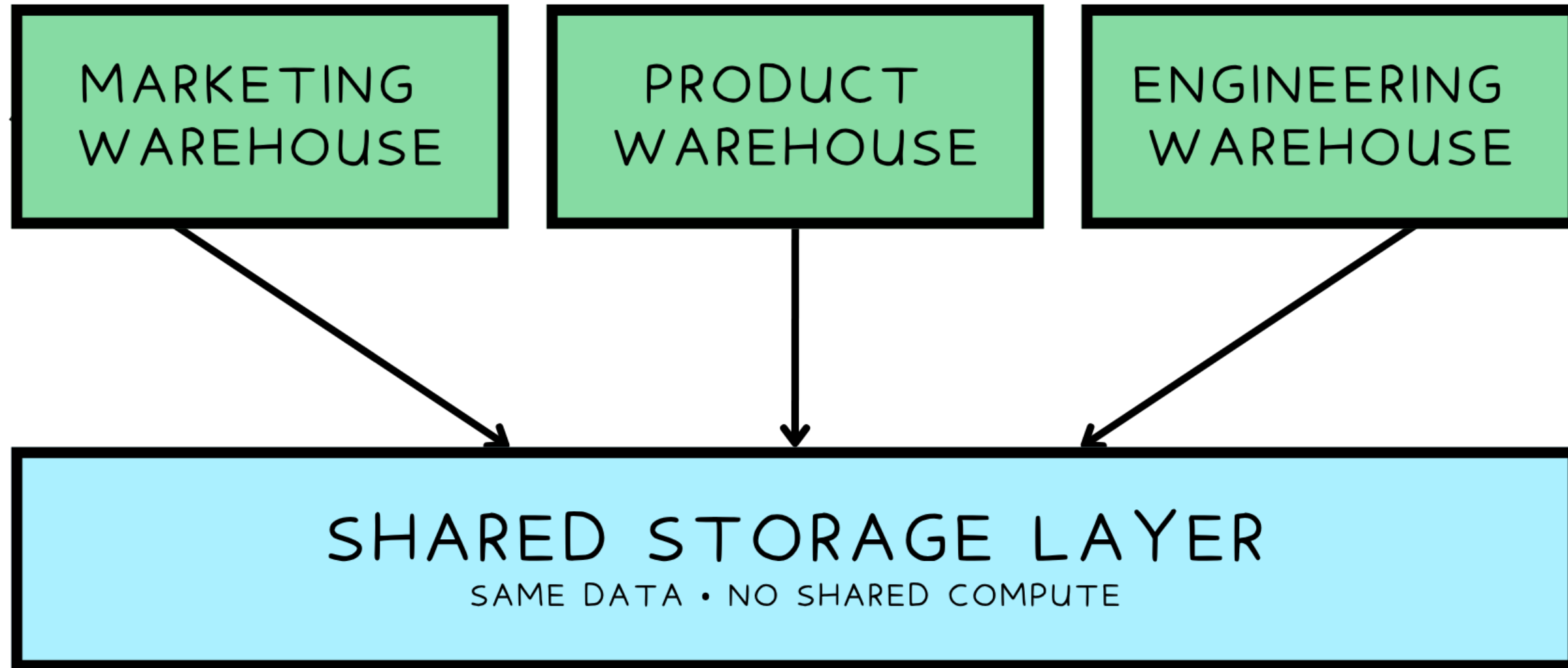


Storage and Compute

BILLED SEPERATELY



Shared Storage Layer

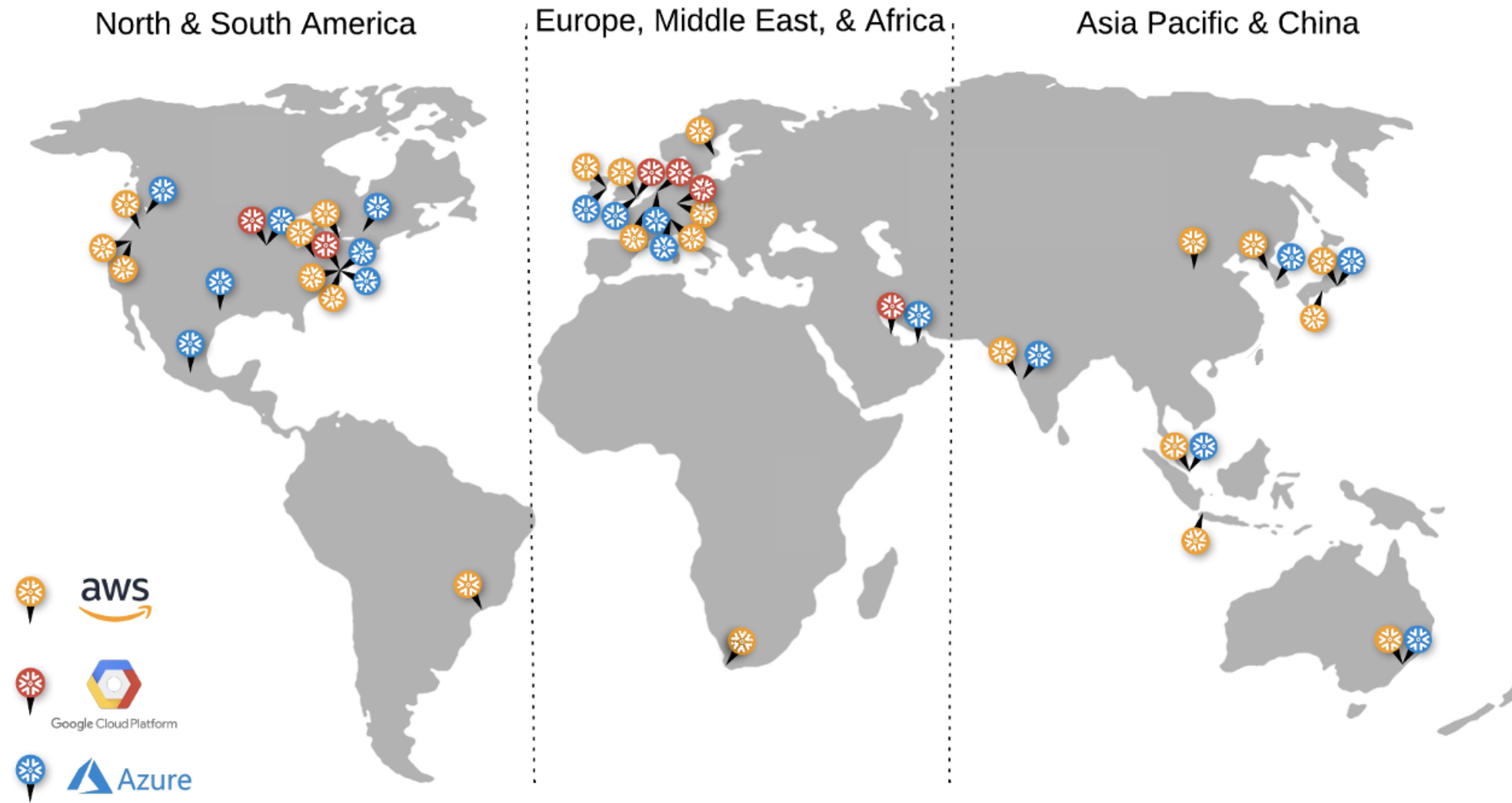


Snowflake Editions

STANDARD	ENTERPRISE	BUSINESS CRITICAL	VIRTUAL PRIVATE SNOWFLAKE (VPS)
			
Complete SQL Data Warehouse Secure Data Sharing across regions /clouds Snowflake Support 24 x 365 1 day of time travel Always-on enterprise-grade encryption in transit and at rest Customer-dedicated virtual warehouses Federated authentication Database Replication External Functions Snowsight Create your own Data Exchange Data Marketplace Access	Standard + Multi-cluster warehouse Up to 90 days of time travel Annual rekey of all encrypted data Materialized Views Search Optimization Service Dynamic Data Masking External Data Tokenization	Enterprise + HIPAA support PCI compliance Tri-Secret Secure using customer-managed keys AWS <u>PrivateLink</u> support Azure Private Link support Google Cloud Private Service Connect support Database Failover and Failback & Client Redirect for business continuity External Functions - AWS API Gateway Private Endpoints support	Business Critical + Customer-dedicated virtual servers wherever the encryption key is in memory Customer-dedicated metadata store

¹ <https://docs.snowflake.com/en/user-guide/intro-key-concepts> <https://docs.snowflake.com/en/user-guide/cost-understanding>

Global Coverage



(Last Updated: April 2026)

¹ <https://docs.snowflake.com/en/user-guide/intro-regions>

Prepare for the SnowPro Core Certification



- SnowPro Core Certification track
 - Snowflake Architecture (**this course**):
Snowflake University Platform Skills Badge
 - Snowflake Management, Governance & Collaboration
 - Data Pipeline Automation in Snowflake
- **Prerequisites**
 - Familiarity with Snowflake and SQL

SnowPro® Core Certification (COF-C03)



- Courses cover the main skills required
 - 1.0 Features & Architecture
 - 2.0 Account Management and Data Governance
 - 3.0 Data Loading, Unloading, and Connectivity
 - 4.0 Performance Optimization, Querying, and Transformation
 - 5.0 Data Collaboration
- Get a **50\$ discount code** once you have completed the SnowPro Core track

Let's practice!
SNOWFLAKE ARCHITECTURE

Ways of Working with Snowflake

SNOWFLAKE ARCHITECTURE



DataCamp

Technical Curriculum Developer,
Snowflake

Three interfaces, three different jobs

SNOWSIGHT

- EXPLORE DATA
- DEBUG QUERIES
- MONITOR PERFORMANCE

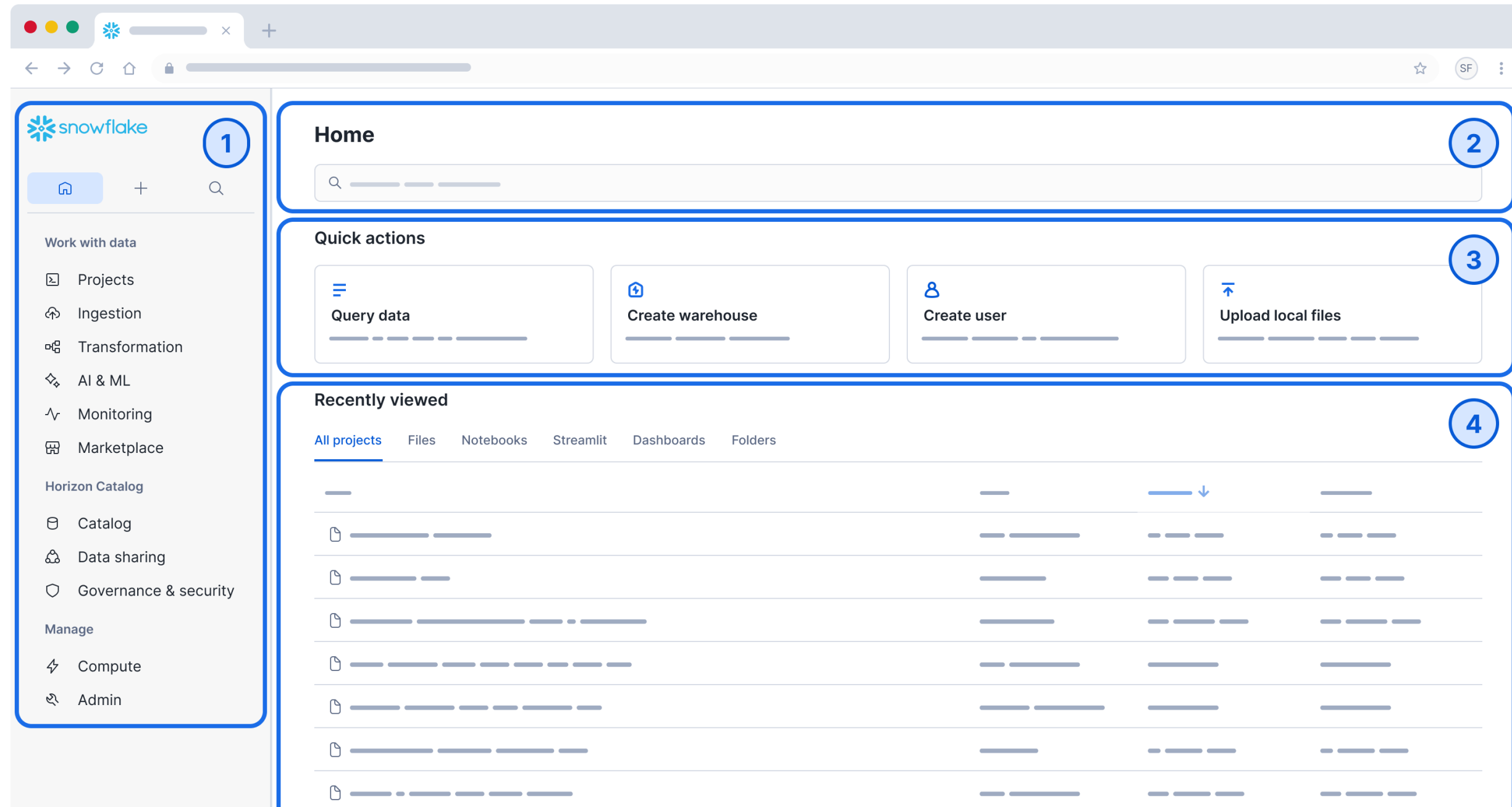
VS CODE

- WRITE CODE
- TEST
- ITERATIVE DEVELOPMENT

CLI

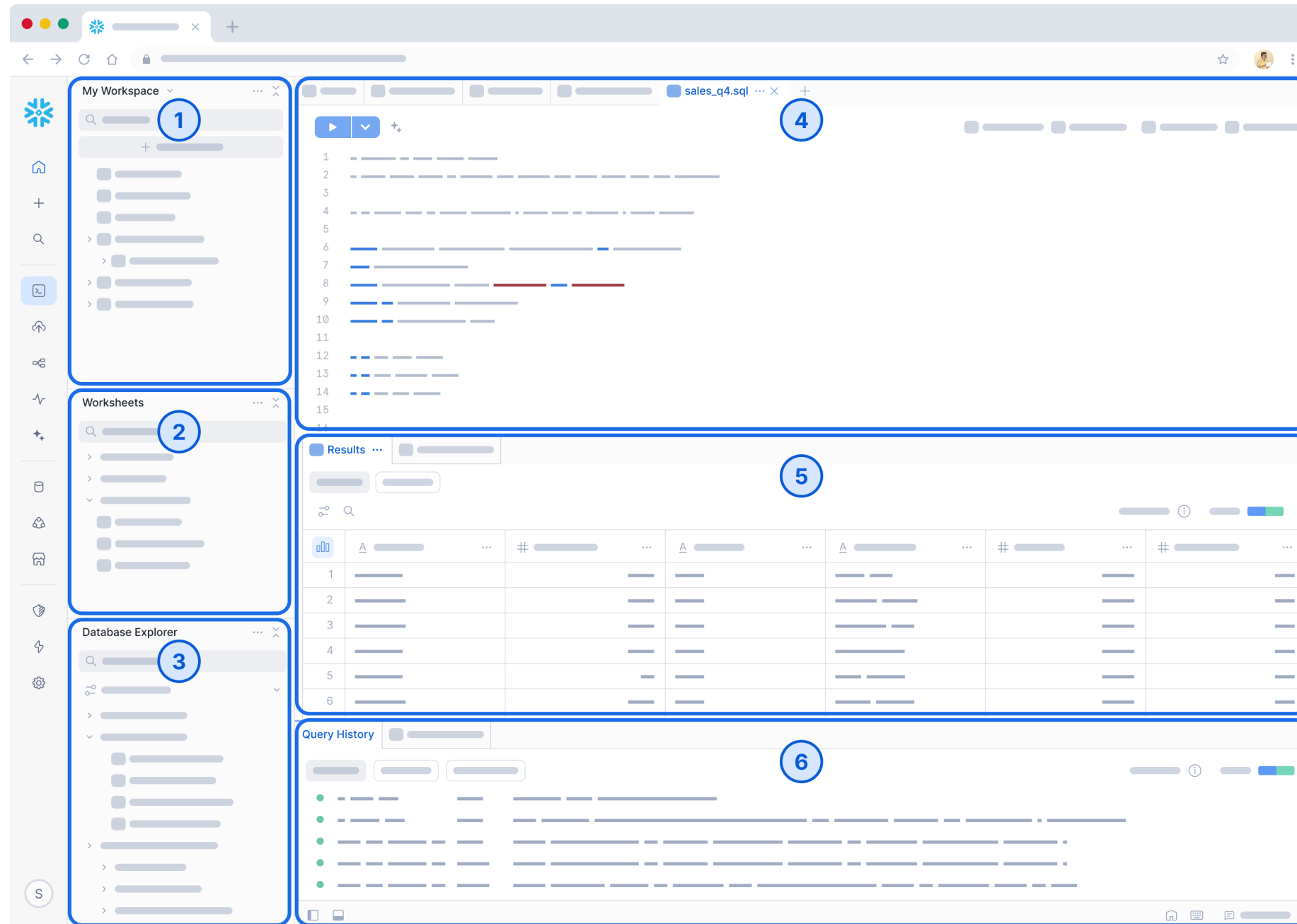
- AUTOMATE WORKFLOWS
- BUILD PIPELINES
- CI/CD

Snowsight



¹ <https://docs.snowflake.com/en/user-guide/ui-snowsight>

Snowsight: Workspaces



¹ <https://docs.snowflake.com/en/user-guide/ui-snowsight/workspaces>

Snowsight: Context

 ACCOUNTADMIN ◦ COMPUTE_WH (X-Small)  Choose database

- Wrong context is the number one reason queries fail
- Role and warehouse are preselected, ensure you set your database and schema yourself.

¹ <https://docs.snowflake.com/en/sql-reference/sql/use>

Snowsight: Query History

The screenshot shows the Snowflake Query History page. On the left is a sidebar with the Snowflake logo and navigation links: Work with data, Projects, Ingestion, Transformation, AI & ML, Monitoring (highlighted), Marketplace, Horizon Catalog, Catalog, Data sharing, Governance & security, Manage, Compute, Postgres, and Admin. The main panel is titled 'Query History' and has two tabs: 'Individual Queries' (selected) and 'Grouped Queries'. Below the tabs are filters: Status (All), User (redacted), Last day, and Filters. On the right are 'Columns' and a refresh button. The table below shows query details with columns: SQL TEXT, QUERY ID, STATUS, USER, WAREHOUSE, DURATION, and STARTED. One query is listed with ID 'c39cb8-0108-6b29-0022-b07f0019700e', status 'Success', and started on 'Apr 10, 2026, 09:56:44'.

	SQL TEXT ↑	QUERY ID ↑	STATUS ↑	USER ↑	WAREHOUSE ↑	DURATION ↑	STARTED ↑
53	SELECT system\$GET_NPS_FEEDBACK_TIMESTAM...	c39cb8-0108-6b29-0022-b07f0019700e	Success	[redacted]	COMPUTE_WH	55ms	Apr 10, 2026, 09:56:44

¹ <https://docs.snowflake.com/en/user-guide/ui-query-history>

Query History: Simplified Example

Monitoring >
Query History

FiltersApply

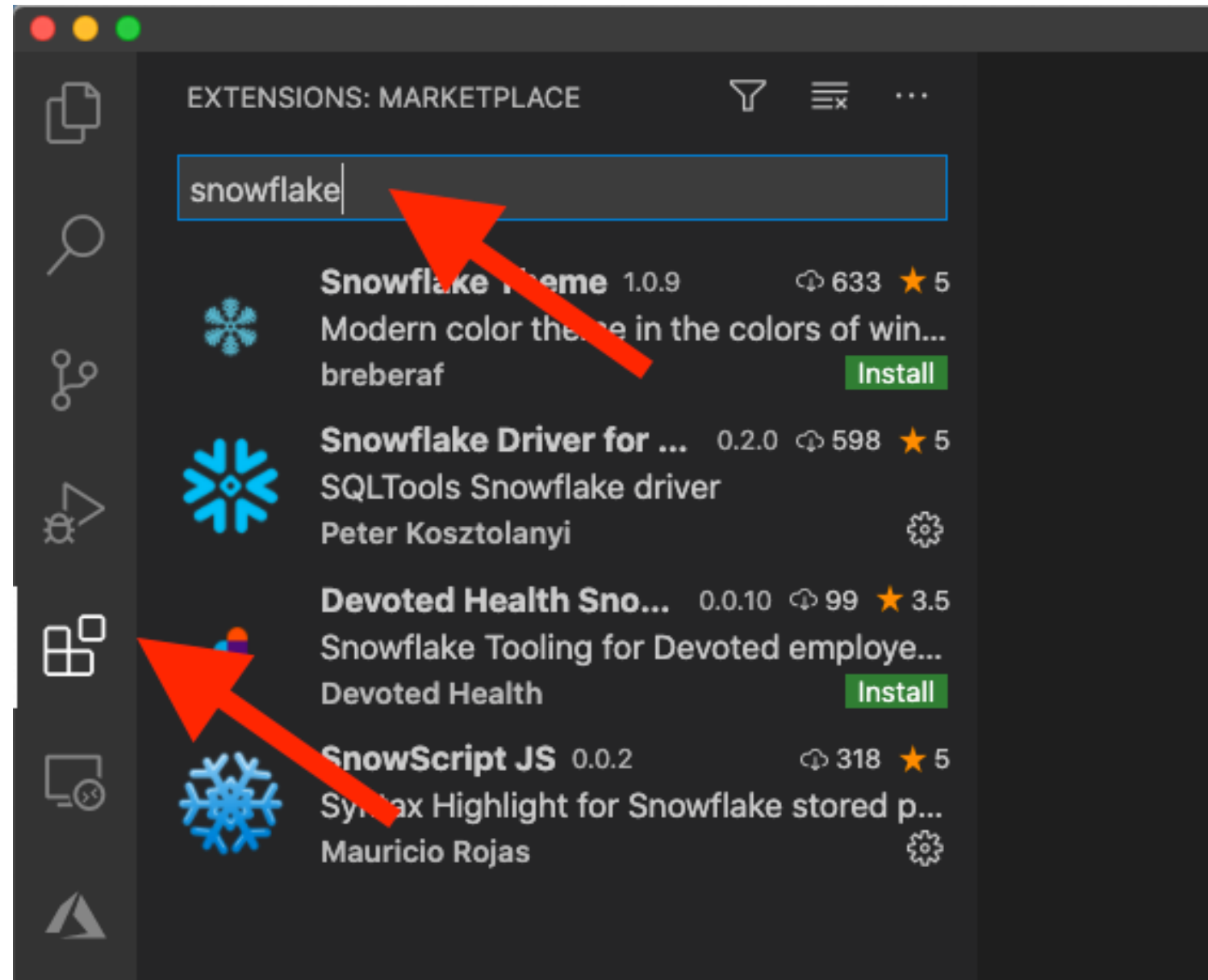
STATUS	QUERY	DURATION	USER	WAREHOUSE
 SUCCESS	SELECT * FROM sales_data WHERE date >= '2024-01-01'	1.2s	analytics_user	COMPUTE_WH
 FAILED	CALL pipeline.transform_daily() Error: Object 'raw_events' does not exist	0.08s	pipeline_svc	ETL_WH
 RUNNING	CREATE TABLE monthly_summary AS SELECT ... FROM orders GROUP BY month	14.3s...	dbt_runner	TRANSFORM_WH
 SUCCESS	SHOW WAREHOUSES	0.3s	admin	SYSTEM
 SUCCESS	COPY INTO raw_events FROM @stage/...	4.7s	loader_svc	INGEST_WH

Pipeline debug
First stop for any failed pipeline run
exact query · time · error

Filters matter
"Don't consume query credits" hides runs
adjust + Apply to see all

What it tracks
status · duration · user
warehouse · full SQL
every query against account

VS Code with the Snowflake extension



1. Install Snowflake Extension
2. Authenticate with Snowflake
3. Execute SQL directly in VS Code

VS Code Extension: Simplified Example

```
bash - validate_pipeline.sh

# CI validation step - snowy_analytics deployment pipeline
snowy-analytics :~/pipelines $ snow sql -q "SELECT COUNT(*) FROM orders WHERE order_date = CURRENT_DATE"
+-----+
| COUNT(*) |
+-----+
| 84291 |      ✓ row count OK (84291 rows)
+-----+

snowy-analytics :~/pipelines $ snow sql -f validate/null_checks.sql --connection prod
✓ null check passed - customer_id: 0 nulls, order_total: 0 nulls

snowy-analytics :~/pipelines $ snow sql -f validate/date_range.sql --connection prod
✓ date range OK - min: 2024-01-01 max: 2024-03-06

snowy-analytics :~/pipelines $ echo $?
0 # all checks passed - pipeline proceeding to deploy

snowy-analytics :~/pipelines $
```

Replaces SnowSQL
Modern CLI for executing
SQL files or inline queries
snow sql -f file.sql | -q "..."

CI/CD native
Fits inside any pipeline,
scheduled job, or script
GitHub Actions · cron · bash

Automated validation
Row counts, nulls, date
ranges – before every deploy
exit 0 = proceed | exit 1 = abort

Snowflake CLI

```
[W654C7KT7W / $ snow]

Usage: snow [OPTIONS] COMMAND [ARGS]...

Snowflake CLI tool for developers [v3.10.0]

Options
  --help          -h          Show this message and exit.
  --version        Shows version of the Snowflake CLI
  --info          Shows information about the Snowflake CLI
  --config-file   FILE       Specifies Snowflake CLI configuration file that should be used
                        [default: None]
  --install-completion  Install completion for the current shell.
  --show-completion    Show completion for the current shell, to copy it or customize the
                        installation.

Commands
  app            Manages a Snowflake Native App
  connection     Manages connections to Snowflake.
  cortex         Provides access to Snowflake Cortex.
  git            Manages git repositories in Snowflake.
  helpers        Helper commands.
  init           Creates project directory from template.
  logs           Retrieves logs for a given object.
  notebook       Manages notebooks in Snowflake.
  object         Manages Snowflake objects like warehouses and stages
  snowpark       Manages procedures and functions.
  spcs           Manages Snowpark Container Services compute pools, services, image registries, and image
  repositories.
  sql            Executes Snowflake query.
  stage          Manages stages.
  streamlit      Manages a Streamlit app in Snowflake.
```

¹ <https://docs.snowflake.com/en/developer-guide/snowflake-cli/index>

CLI: Simplified Example

```
bash - validate_pipeline.sh

# CI validation step - snowy_analytics deployment pipeline
snowy-analytics :~/pipelines $ snow sql -q "SELECT COUNT(*) FROM orders WHERE order_date = CURRENT_DATE"
+-----+
| COUNT(*) |
+-----+
| 84291 |      ✓ row count OK (84291 rows)
+-----+

snowy-analytics :~/pipelines $ snow sql -f validate/null_checks.sql --connection prod
✓ null check passed - customer_id: 0 nulls, order_total: 0 nulls

snowy-analytics :~/pipelines $ snow sql -f validate/date_range.sql --connection prod
✓ date range OK - min: 2024-01-01 max: 2024-03-06

snowy-analytics :~/pipelines $ echo $?
0 # all checks passed - pipeline proceeding to deploy

snowy-analytics :~/pipelines $
```

Replaces SnowSQL
Modern CLI for executing
SQL files or inline queries
`snow sql -f file.sql | -q "..."`

CI/CD native
Fits inside any pipeline,
scheduled job, or script
GitHub Actions · cron · bash

Automated validation
Row counts, nulls, date
ranges – before every deploy
exit 0 = proceed | exit 1 = abort

SNOWFLAKE · CLI · AUTOMATED PIPELINE VALIDATION

Let's practice!
SNOWFLAKE ARCHITECTURE

Snowflake Object Hierarchy

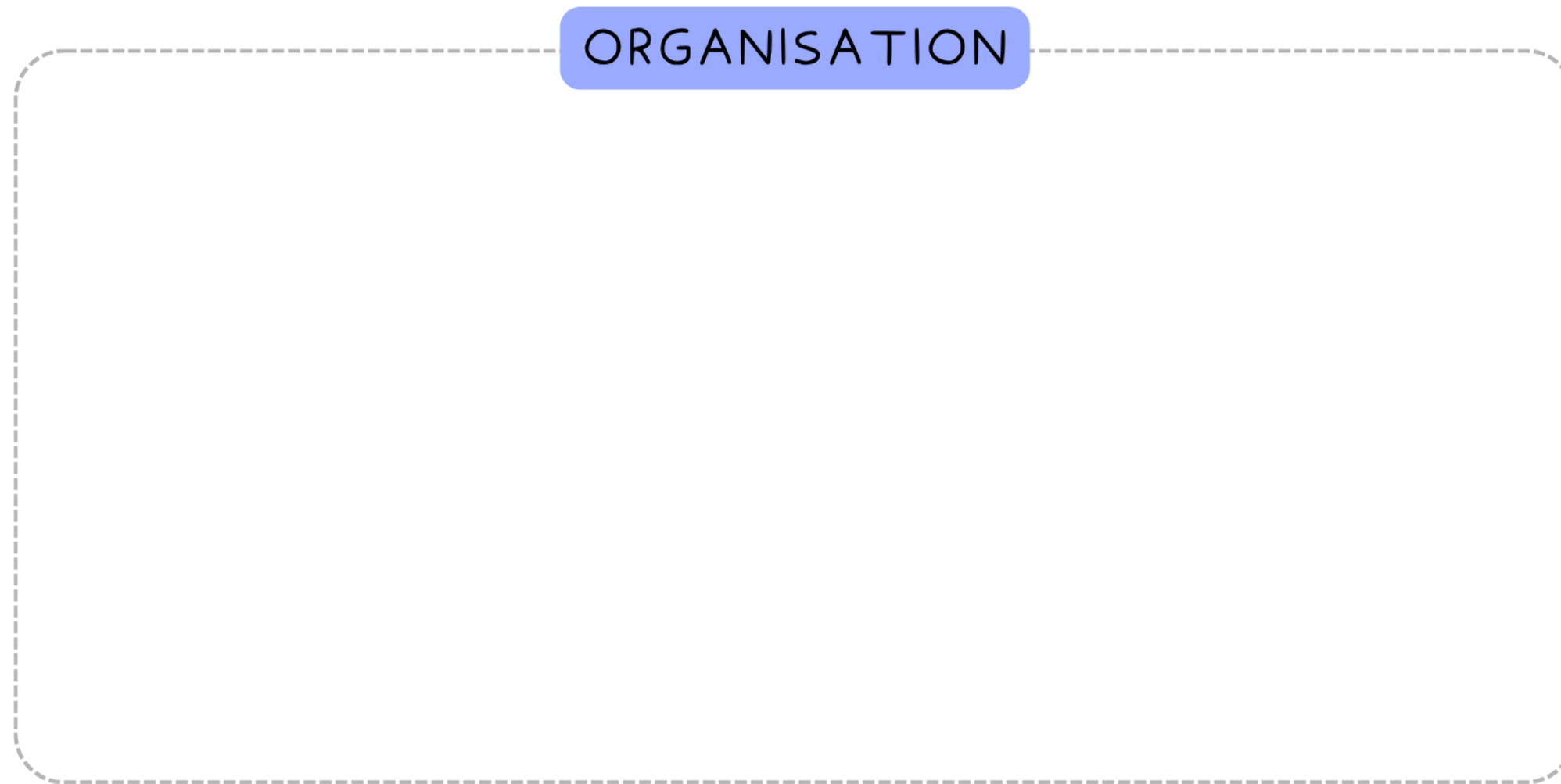
SNOWFLAKE ARCHITECTURE



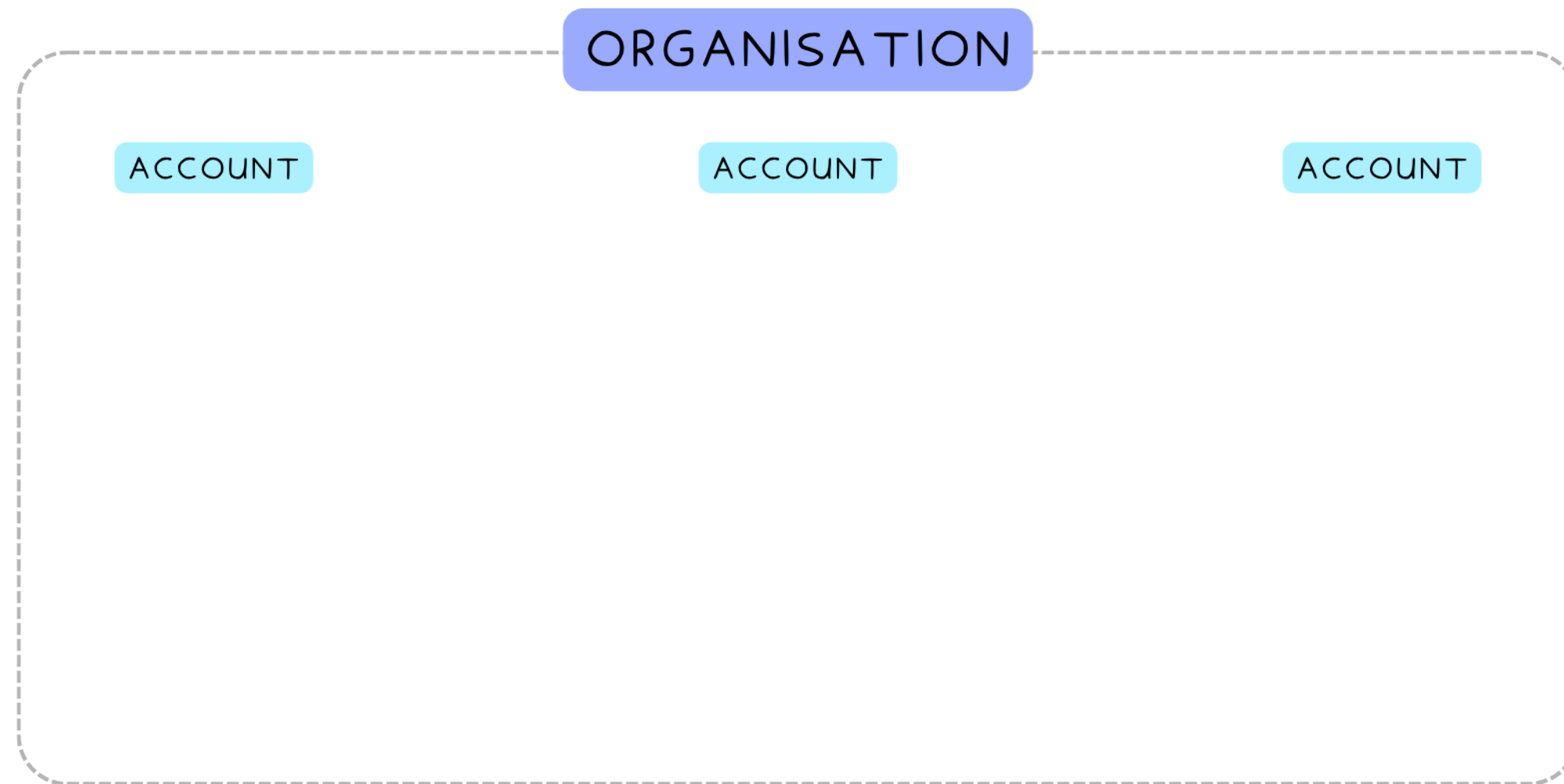
Emily Melhuish

Technical Curriculum Developer,
Snowflake

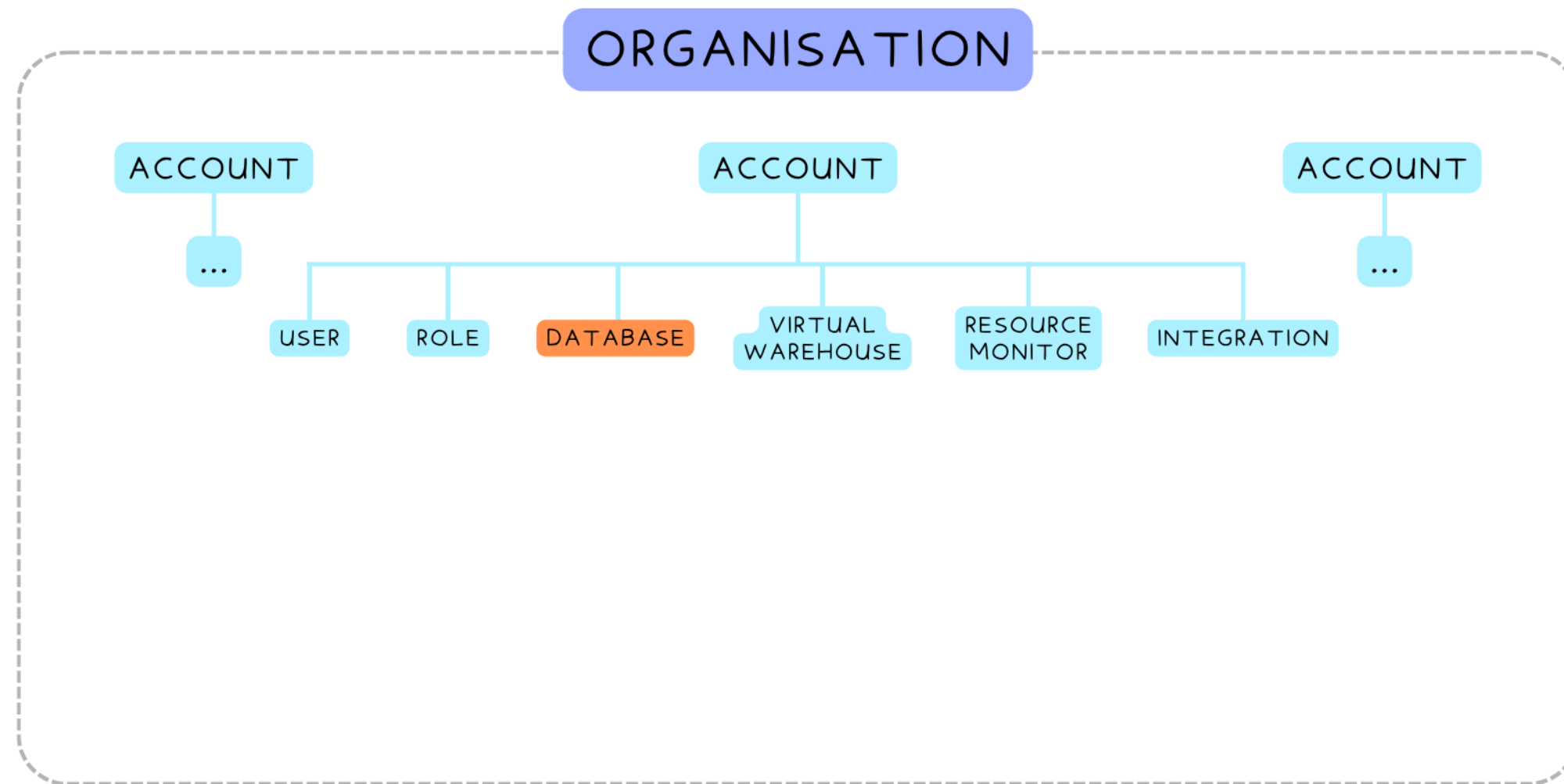
Understanding the Object Hierarchy



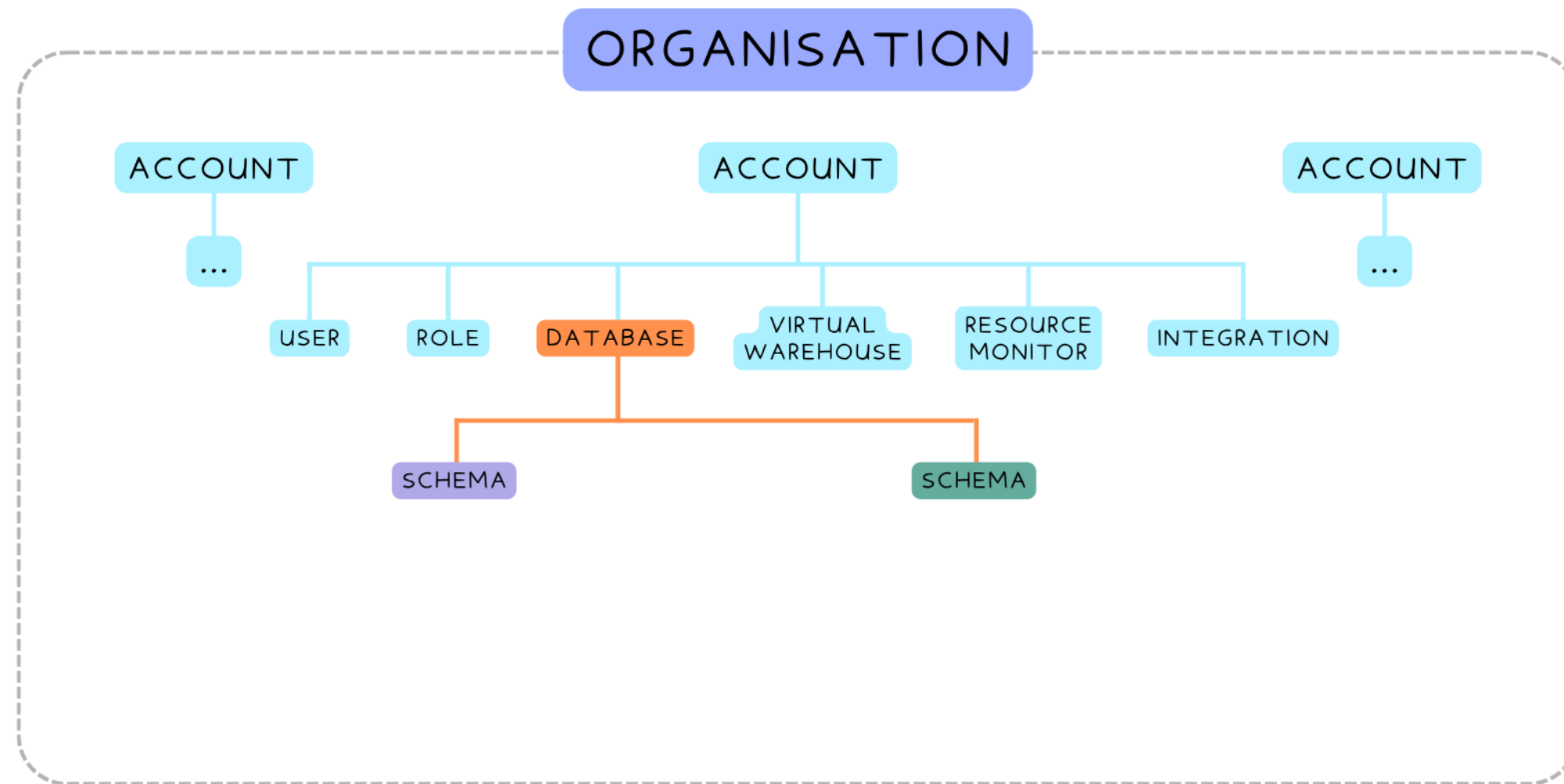
Understanding the Object Hierarchy



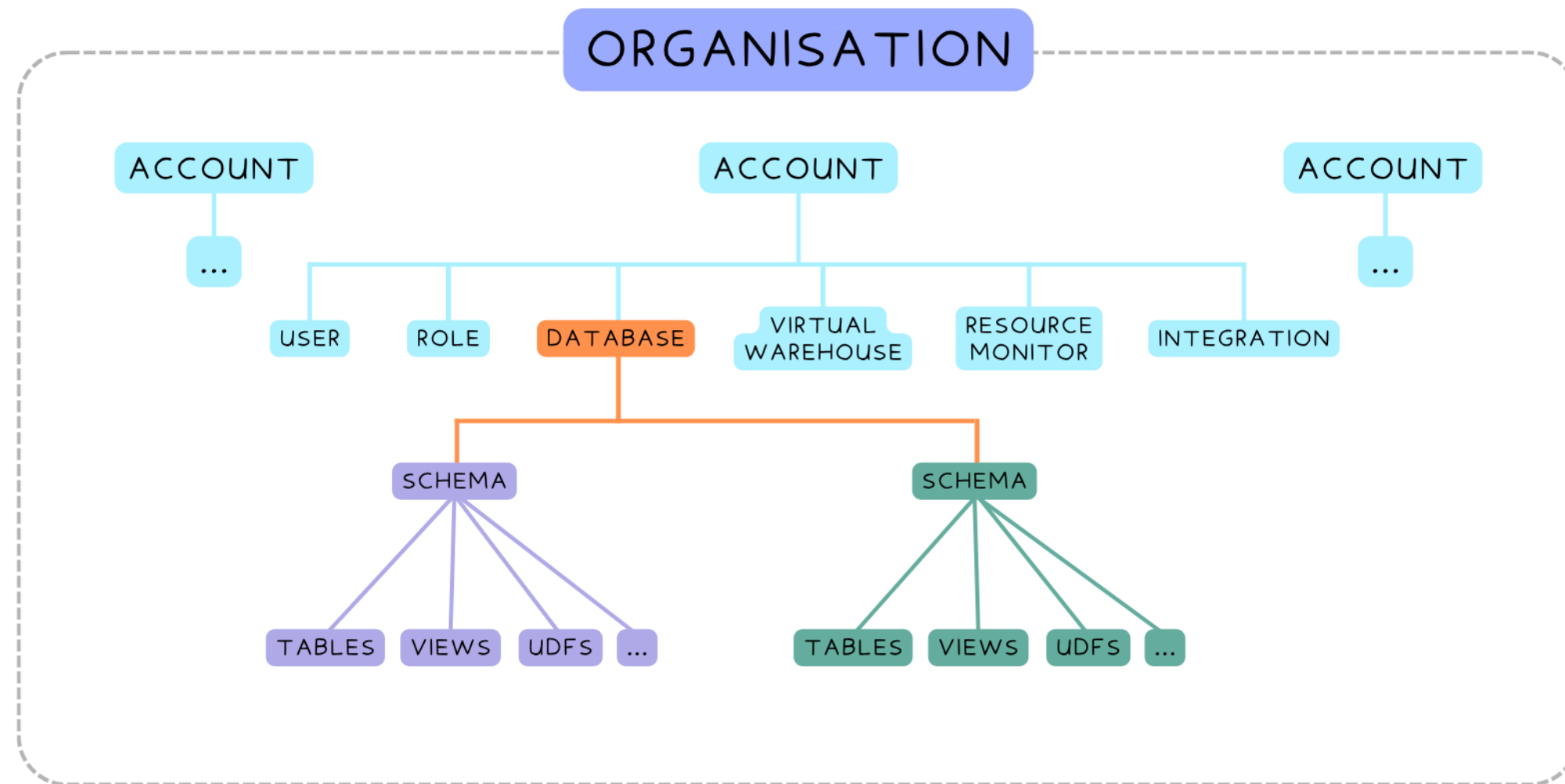
Understanding the Object Hierarchy



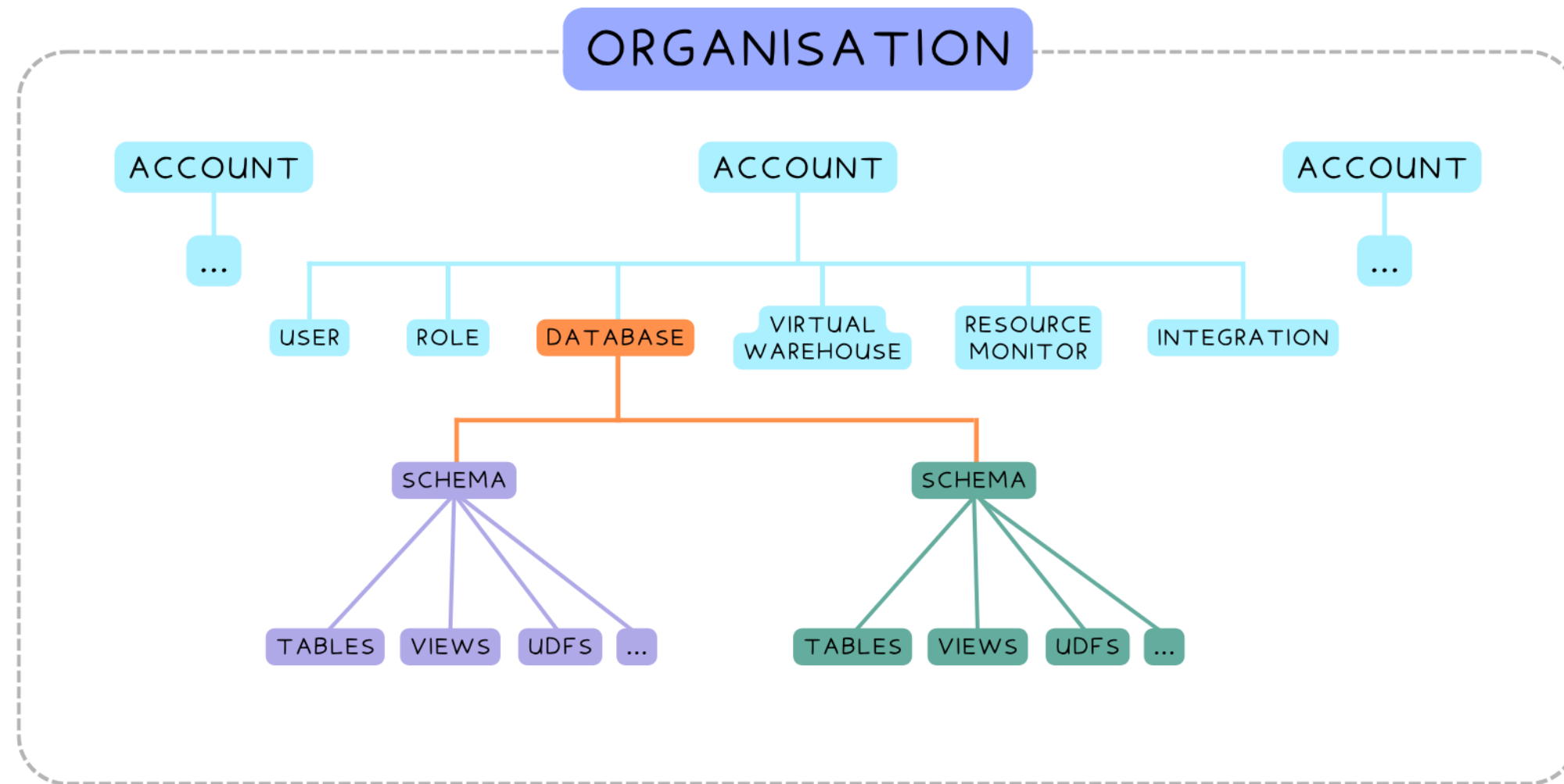
Understanding the Object Hierarchy



Understanding the Object Hierarchy



Account-level vs Database-level Objects



¹ <https://docs.snowflake.com/en/user-guide/security-access-control-overview>

Context

Role → Warehouse → Database → Schema

Set Session Context

- `USE ROLE`
- `USE WAREHOUSE`
- `USE DATABASE`
- `USE SCHEMA`

Check Session Context

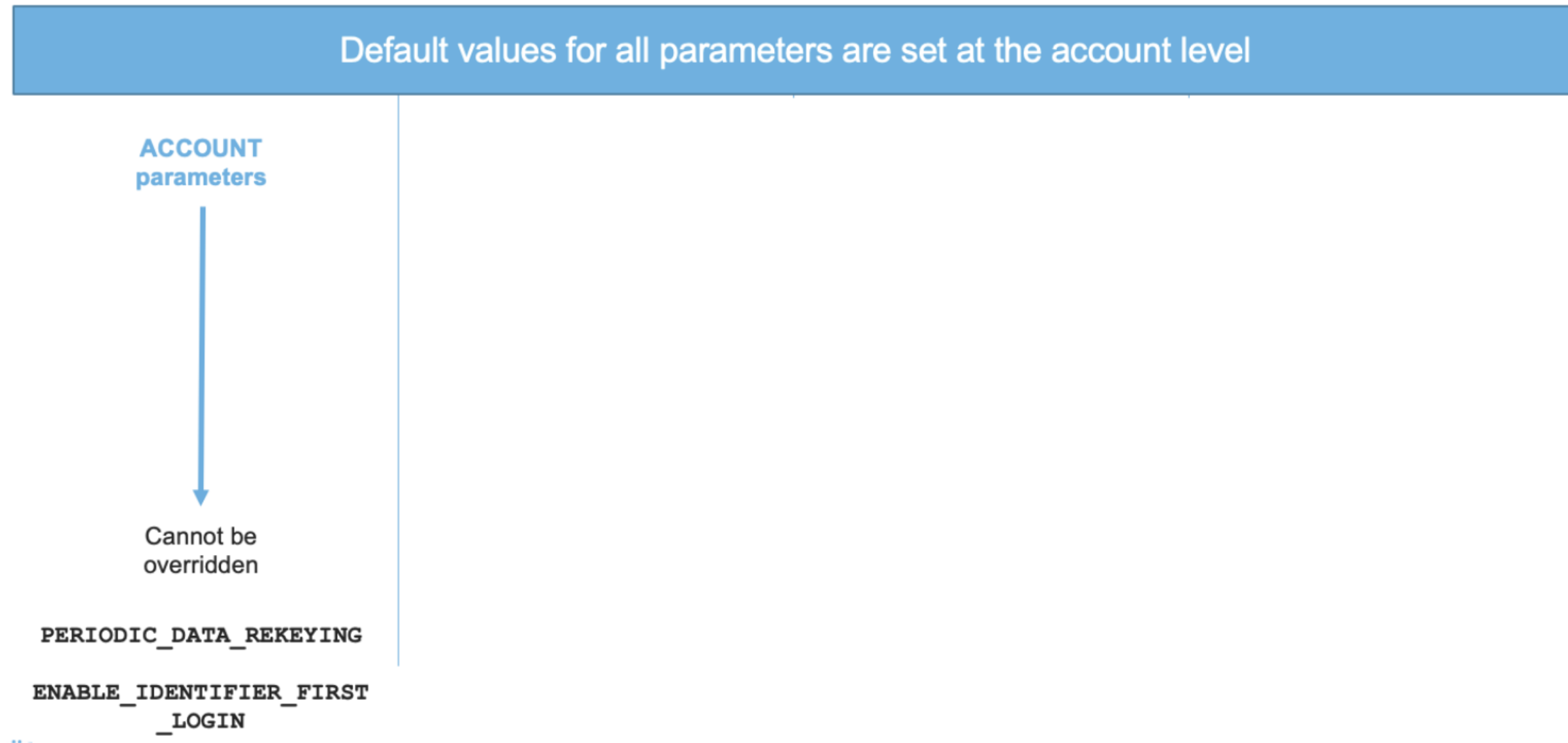
- `CURRENT_ROLE()`
- `CURRENT_WAREHOUSE()`
- `CURRENT_DATABASE()`
- `CURRENT_SCHEMA()`

Session Variables

- Store and re-use values
- Create: `SET min_users = 100`
- Use: `SELECT $min_users;`

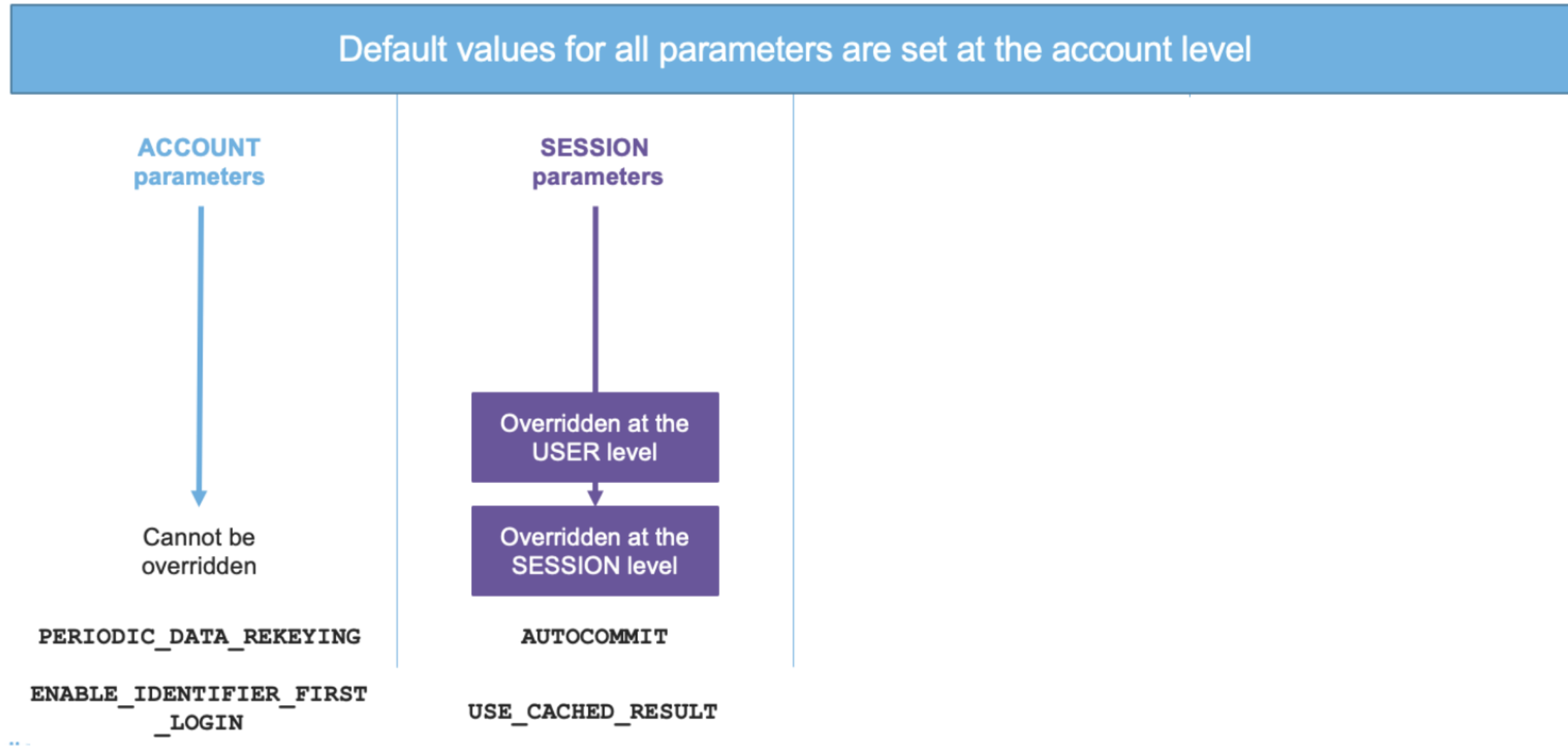
¹ <https://docs.snowflake.com/en/sql-reference/sql/set>

Parameter hierarchy: Account Parameters



¹ <https://docs.snowflake.com/en/sql-reference/parameters>

Session Parameters



Virtual Warehouse Parameters

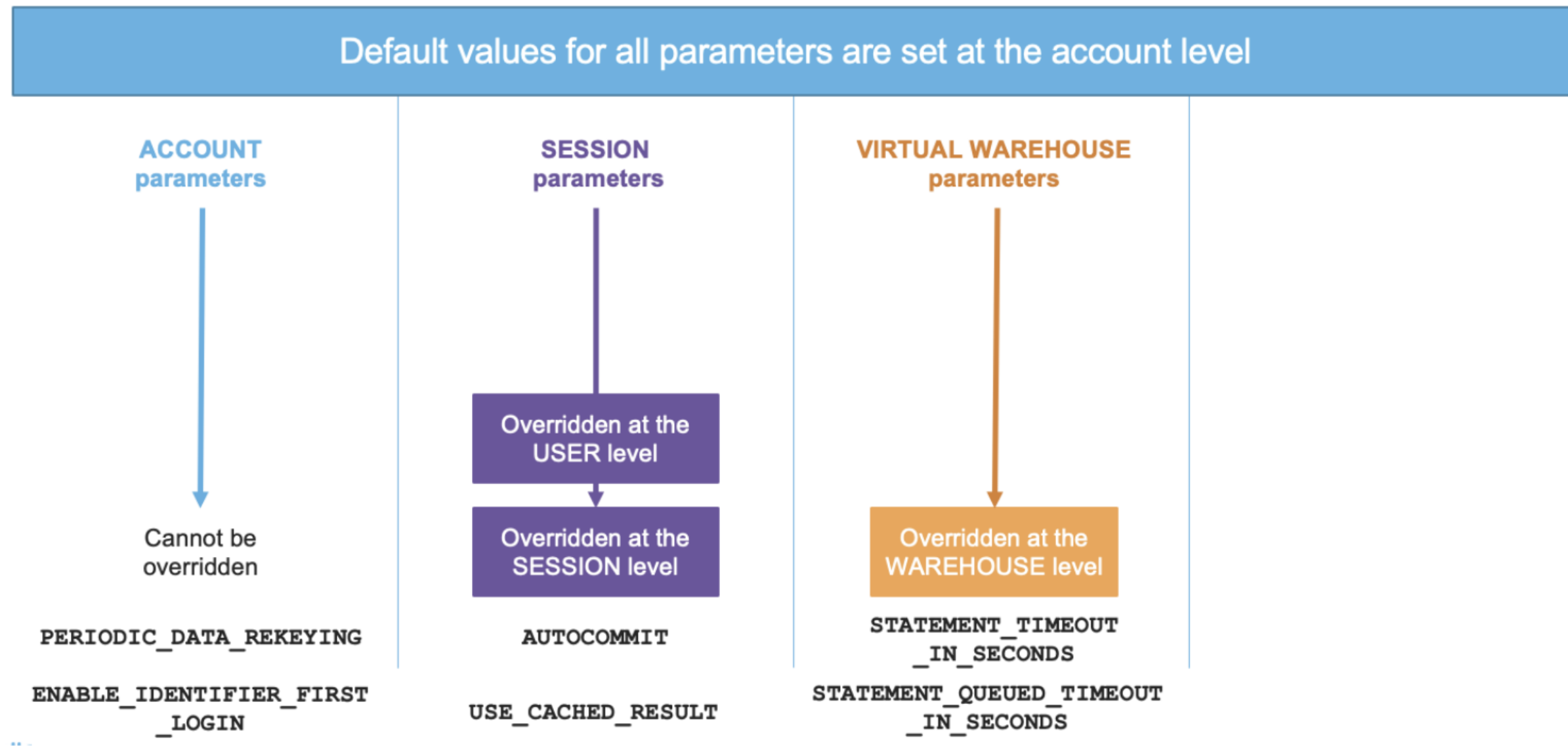
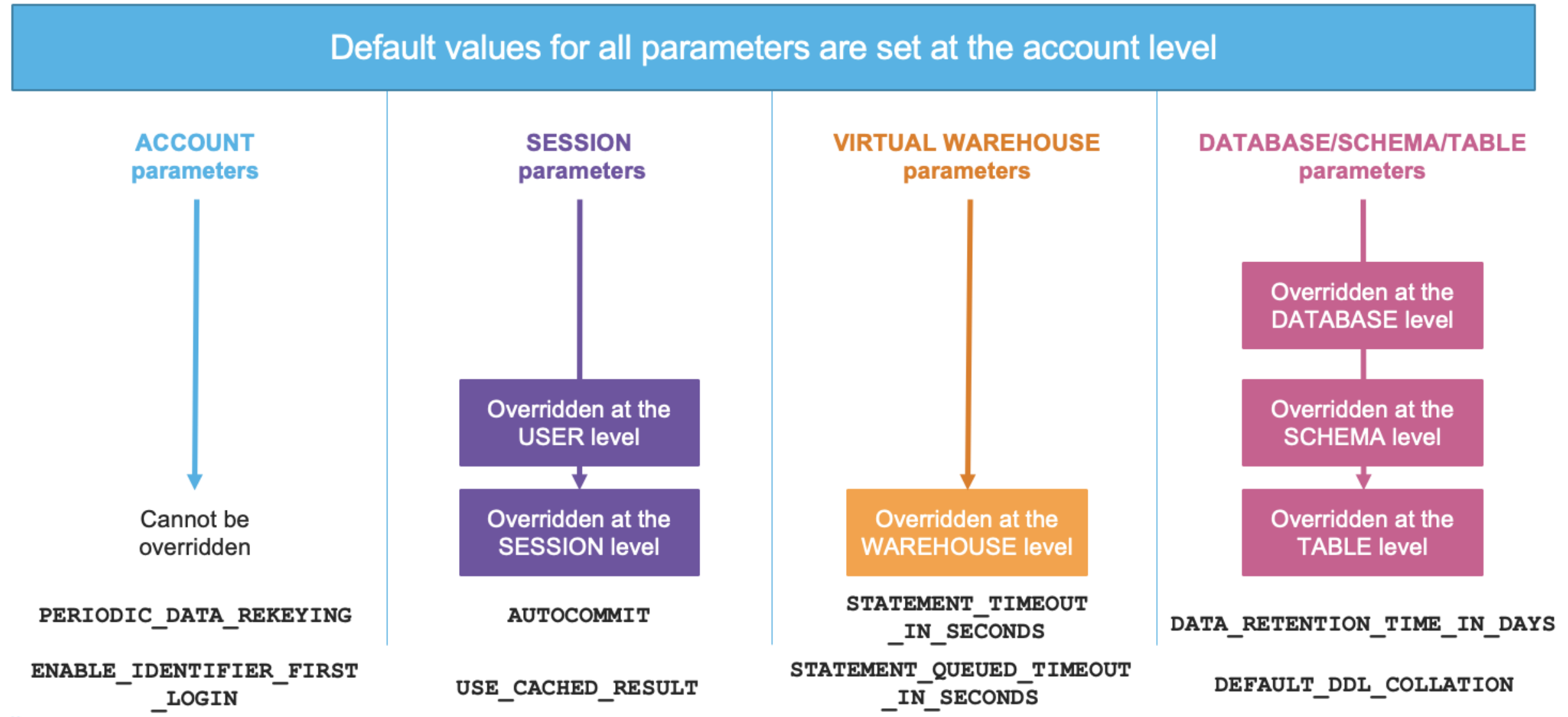


Table Parameters



Let's practice!
SNOWFLAKE ARCHITECTURE